

KIMBERLY C. EVANGELISTA

BSCPE (LADDERIZED)

Code with explanations

```
esp32 | Arduino IDE 2.3.6
File Edit Sketch Tools Help

ESP32P4 Dev Module

esp32.ino
1 #include <ESP32Servo.h> // Include Servo library for ESP32
2
3 // ESP32 Pin setup
4 const int ldrPin = 34; // ADC-capable pin for LDR
5 const int ledPin = 13; // GPIO for LED
6 const int trigPin = 18; // GPIO for Ultrasonic Trigger
7 const int echoPin = 19; // GPIO for Ultrasonic Echo
8 const int servoPin = 5; // PWM-capable GPIO for Servo
9
10 // Lumen and Distance thresholds
11 const float lumenThreshold = 20.0;
12 const float distanceThreshold = 20.0; // cm
13
14 Servo myServo; // Create servo object
15
16 void setup() {
17   pinMode(ledPin, OUTPUT);
18   pinMode(trigPin, OUTPUT);
19   pinMode(echoPin, INPUT);
20
21   Serial.begin(115200);
22
23   myServo.attach(servoPin); // Attach servo to GPIO pin
24   myServo.write(90); // Start at neutral position
25 }
26
27 void loop() {
28   // 1. Read LDR value (ESP32 ADC range is 0-4095)
```

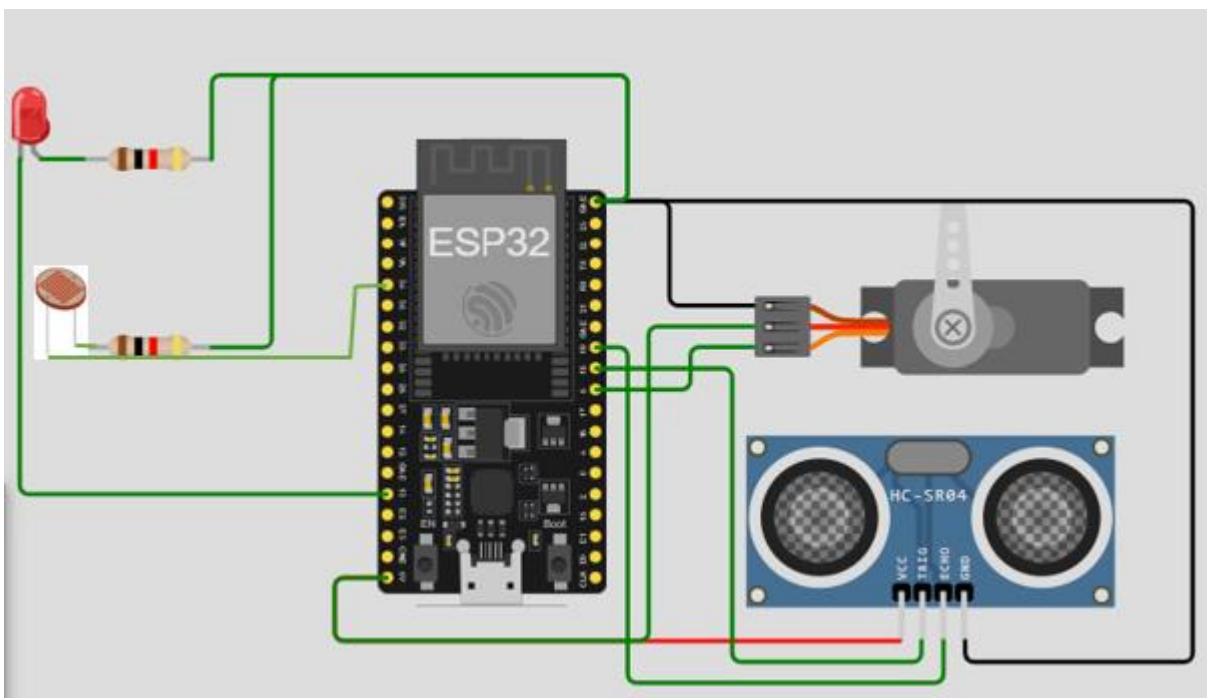
```
ESP32P4 Dev Module

esp32.ino
28 // 1. Read LDR value (ESP32 ADC range is 0-4095)
29 int analogValue = analogRead(ldrPin);
30 float voltage = analogValue * (3.3 / 4095.0); // ESP32 is 3.3V logic
31
32 // Estimate resistance of LDR (using 10k pull-down resistor)
33 float resistance = (3.3 - voltage) * 10000.0 / voltage;
34
35 // Estimate lux (rough formula)
36 float lux = 500 / (resistance / 1000.0);
37
38 // 2. Measure distance
39 float measuredDistance = measureDistanceCM();
40
41 // 3. Apply calibration
42 float correctedDistance = 0.95 * measuredDistance - 0.5;
43
44 // 4. Debug output
45 Serial.print("Lux: ");
46 Serial.print(lux);
47 Serial.print(" | Corrected Distance: ");
48 Serial.print(correctedDistance);
49 Serial.println(" cm");
50
51 // 5. LED control logic
52 if (lux < lumenThreshold && correctedDistance <= distanceThreshold) {
53   digitalWrite(ledPin, HIGH);
54 } else {
55   digitalWrite(ledPin, LOW);
```

```
ESP32P4 Dev Module

esp32.ino
50
51 // 5. LED control logic
52 if (lux < lumenThreshold && correctedDistance <= distanceThreshold) {
53   digitalWrite(ledPin, HIGH);
54 } else {
55   digitalWrite(ledPin, LOW);
56 }
57
58 // 6. Servo motor control logic (map distance to angle)
59 int angle = map(correctedDistance, 5, 30, 0, 180); // Map 5-30 cm to 0-180°
60 angle = constrain(angle, 0, 180); // Clamp within valid range
61 myServo.write(angle); // Set servo position
62
63 delay(1000);
64 }
65
66 // Function to measure distance using HC-SR04 on ESP32
67 float measureDistanceCM() {
68   digitalWrite(trigPin, LOW);
69   delayMicroseconds(2);
70   digitalWrite(trigPin, HIGH);
71   delayMicroseconds(10);
72   digitalWrite(trigPin, LOW);
73
74   long duration = pulseIn(echoPin, HIGH, 30000); // 30 ms timeout
75   float distance = duration * 0.034 / 2.0;
76   return distance;
77 }
```

Diagram



Multi-Sensor ESP32 System/ Automated Street Light

Evaluation of System Performance

The developed multi-sensor data acquisition system effectively composed different modules- an LDR in act as light intensity sensor, Ultrasonic sensor with respect to proximate detecting, light emitting diode (L.E.D) for visual indication and Servo Motor for mechanical interaction. The system was breadboarded with an ESP32, which provided plenty of analog/digital I/O pins and built-in Wi-Fi (the latter which was not used for the challenge).

The light-sensing subsystem, using a voltage divider with a 10kΩ resistor and LDR, offered steady detection of dark environments. The estimated lux was obtained by a simple inverse resistance model. It was not very accurate (as it was not calibrated according to an actual lux meter), but was good enough to switch LEDs on at a certain threshold.

The ultrasonic sensor (HC-SR04) provided consistent distance readings within the practical range of 5–30 cm. However, some noise was observed, especially on reflective or soft surfaces. A basic linear curve fitting equation ($\text{correctedDistance} = 0.95 * \text{measured} - 0.5$) helped reduce systematic error and improve realism in the sensor readings.

The servo motor effectively responded to distance input, mapping values to angles between 0° and 180°. This gave a clear mechanical response that visually confirmed object proximity. The LED acted as a secondary indicator, illuminating only when both the light level was low and an object was close.

Limitations and Potential Improvements

While the system achieved its basic goals, there were several limitations:

- **Sensor Noise:** Both the LDR and HC-SR04 are susceptible to environmental noise (e.g., ambient light shifts or ultrasonic echoes off angled surfaces). Implementing a **moving average filter** would help smooth out such fluctuations.
- **Lack of Calibration:** The LDR's lux calculation is an approximation. A more precise approach would be to calibrate against a real light meter or use a digital sensor like the BH1750 for accurate lux values.
- **Fixed Thresholds:** The system used hardcoded light and distance thresholds. Adding a **user-adjustable potentiometer** or **touch input** could allow dynamic control over system sensitivity.
- **Servo Power Supply:** The servo draws significant current; if powered directly from the ESP32, brownout or instability may occur. Using an **external power supply with common ground** would ensure more reliable operation.
- **No Feedback Loop:** The servo position is based purely on distance input. A more advanced version could include **PID control** or use feedback from the servo (if available) to improve accuracy.

Reflection and Learnings

This was an invaluable source of practical experience for embedded systems design and integration. A few things I learned: - Interfacing and syncing multiple sensors and servo-motors to an ESP32 microcontroller. Key takeaways include:

- Analogue to digital conversion on ESP32 and Arduino parts 1-7.
- Using very simple electronic principles to build actual working circuits to use as sensors.
- Curve fitting for a better accuracy of the sensor - useful for real life calibration.
- Translating the real world into actionable output like LED states or servo angles.
- And, of course, the importance of writing modular and readable code, with functions such as `measureDistanceCM()` in order to reach that sweet, sweet clarity and reusability.

In the end, this project emphasized the role of both reliable hardware as well as logical software when it comes to developing a real-time and responsive embedded system. It also showed how some of the most basic building blocks, through clever logic, can form a system with easily understandable applications in automation, robotics and environmental monitoring.